

```
# Step 1: Install Required Libraries
!pip install -q scikit-learn matplotlib seaborn tensorflow pandas

# Step 2: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.ensemble import RandomForestClassifier
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

#Scikit-learn is used for traditional ML models like Random Forest and evaluation metrics.

#TensorFlow/Keras is used to build an Artificial Neural Network (ANN).

# Early stopping helps prevent overfitting during training.

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00519/heart_failure_clinical_records_dataset.csv"
df = pd.read_csv(url)
print("Dataset Sample:")
print(df.head())

#We're loading a real-world dataset related to heart failure
#clinical records from the UCI Machine Learning Repository.
```

Dataset Sample:

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	\
0	75.0	0	582	0	20	
1	55.0	0	7861	0	38	
2	65.0	0	146	0	20	
3	50.0	1	111	0	20	
4	65.0	1	160	1	20	

	high_blood_pressure	platelets	serum_creatinine	serum_sodium	sex	\
0	1	265000.00	1.9	130	1	
1	0	263358.03	1.1	136	1	
2	0	162000.00	1.3	129	1	
3	0	210000.00	1.9	137	1	
4	0	327000.00	2.7	116	0	

	smoking	time	DEATH_EVENT
0	0	4	1
1	0	6	1
2	1	7	1
3	0	7	1
4	0	8	1

```
# Step 4: Data Preprocessing
X = df.drop('DEATH_EVENT', axis=1)
y = df['DEATH_EVENT']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

#DEATH_EVENT is the target label (1 = patient died, 0 = survived).

#All other columns are features (X).

#Features are standardized using StandardScaler for better ANN performance.

#We split the data into 80% training and 20% testing sets.

# Step 5: Machine Learning - Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_preds = rf.predict(X_test)
print("\n Random Forest Classifier Report:")
print(classification_report(y_test, rf_preds))
print("Random Forest Accuracy:", accuracy_score(y_test, rf_preds))
```

Here we are training a Random Forest, a powerful ensemble ML algorithm, and evaluate it:
 # The print classification shows the model's precision, recall, and F1-score.



```
Random Forest Classifier Report:
      precision    recall  f1-score   support

     0       0.72      0.94      0.81      35
     1       0.86      0.48      0.62      25

 accuracy          0.75      60
 macro avg          0.79      60
 weighted avg       0.78      60
```

Random Forest Accuracy: 0.75

```
# Step 6: ANN Model with Dropout + EarlyStopping
model = Sequential()
model.add(Dense(16, input_dim=X.shape[1], activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(8, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

#The model has 3 layers:

#First layer: 16 neurons

#Second layer: 8 neurons

#Output: 1 neuron with sigmoid activation for binary classification.

#Dropout layers help prevent overfitting.

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argumer
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Add EarlyStopping
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
```

```
# Train the model
history = model.fit(
X_train, y_train,
epochs=100,
batch_size=16,
validation_data=(X_test, y_test),
callbacks=[early_stop],
verbose=0
)
```

```
# Evaluate the model
loss, ann_acc = model.evaluate(X_test, y_test, verbose=0)
ann_preds = (model.predict(X_test) > 0.5).astype("int32")
print("\n🤖 Artificial Neural Network Report:")
print(classification_report(y_test, ann_preds))
print("ANN Accuracy:", ann_acc)
```

2/2 ————— 0s 105ms/step

```
🤖 Artificial Neural Network Report:
      precision    recall  f1-score   support

     0       0.71      0.91      0.80      35
     1       0.80      0.48      0.60      25

 accuracy          0.73      60
 macro avg          0.76      60
 weighted avg       0.75      60
```

ANN Accuracy: 0.7333333492279053

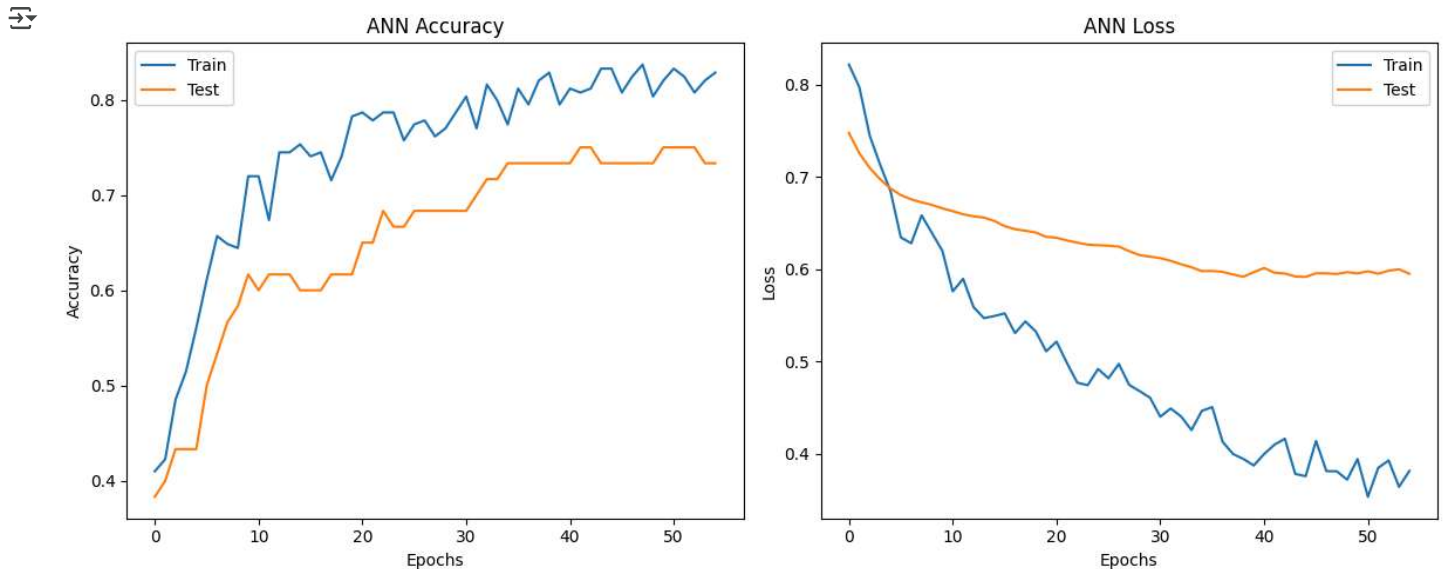
```
# Step 7: Visualize ANN Training
plt.figure(figsize=(12,5))
```

```

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Test')
plt.title('ANN Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
22
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Test')
plt.title('ANN Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()

```

We plot accuracy and loss trends for training and validation sets to visualize performance.



Step 8: Take User Input & Predict

```

def get_user_input():
    print("Please enter the following patient information:")

    def get_integer_input(prompt, valid_values):
        while True:
            try:
                value = int(input(prompt))
                if value in valid_values:
                    return value
            except:
                print(f"Invalid input. Please enter one of the following values: {valid_values}")
        except ValueError:
            print("Invalid input. Please enter an integer.")

    age = float(input("Age (years): "))
    anaemia = get_integer_input("Anaemia (1 = Yes, 0 = No): ", [0, 1])
    cpk = float(input("Creatinine Phosphokinase (e.g., 250): "))
    diabetes = get_integer_input("Diabetes (1 = Yes, 0 = No): ", [0, 1])
    ef = float(input("Ejection Fraction (e.g., 38): "))
    hbp = get_integer_input("High Blood Pressure (1 = Yes, 0 = No): ", [0, 1])
    platelets = float(input("Platelets count (e.g., 263358.03): "))
    sc = float(input("Serum Creatinine (e.g., 1.1): "))
    ss = float(input("Serum Sodium (e.g., 130): "))
    sex = get_integer_input("Sex (1 = Male, 0 = Female): ", [0, 1])
    smoking = get_integer_input("Smoking (1 = Yes, 0 = No): ", [0, 1])
    time = int(input("Follow-up period in days (e.g., 4): "))
    return [age, anaemia, cpk, diabetes, ef, hbp, platelets, sc, ss, sex, smoking, time]

def predict_person(model, scaler, input_data):

```

```

input_data = np.array(input_data).reshape(1, -1)
input_scaled = scaler.transform(input_data)
prediction = model.predict(input_scaled)[0][0]
print("\n🔴 Prediction Probability:", round(prediction, 3))
if prediction >= 0.5:
    print("⚠️ High Risk of Heart Disease")
else:
    print("✅ Low Risk of Heart Disease")

# Run the prediction
user_data = get_user_input()
predict_person(model, scaler, user_data)

#This function asks the user to manually input patient data
#from 12 clinical fields like age, blood pressure, diabetes, etc.

#Takes the input data, scales it using the same StandardScaler, and makes prediction using the ANN model.

##Displays risk level based on prediction probability.

🔄 Please enter the following patient information:
Age (years): 24
Anaemia (1 = Yes, 0 = No): 1
Creatinine Phosphokinase (e.g., 250): 260
Diabetes (1 = Yes, 0 = No): 1
Ejection Fraction (e.g., 38): 39
High Blood Pressure (1 = Yes, 0 = No): 1
Platelets count (e.g., 263358.03): 239000
Serum Creatinine (e.g., 1.1): 1.2
Serum Sodium (e.g., 130): 145
Sex (1 = Male, 0 = Female): 0
Smoking (1 = Yes, 0 = No): 1
Follow-up period in days (e.g., 4): 7
1/1 ————— 0s 47ms/step

🔴 Prediction Probability: 0.058
✅ Low Risk of Heart Disease
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Standard
warnings.warn(

```

This project compares Random Forest vs ANN on heart disease prediction:

Shows evaluation using accuracy and classification report.

ANN is made more robust with Dropout and EarlyStopping.

We added real-time prediction by taking user input and evaluating the risk.

Start coding or [generate](#) with AI.